

Additional I

Shortest Subsequence

```
#include

using namespace std;
const int maxN = 1e6+5;

int N, cnt;
char S[maxN], ch[4] = {'A', 'C', 'G', 'T'};
bool has[4];
vector segs;
map mp;

void init(){
    mp['A'] = 0;
    mp['C'] = 1;
    mp['G'] = 2;
    mp['T'] = 3;
}

int main(){
    init();
    scanf(" %s", S);

    N = (int) strlen(S);
    for(int i = 0; i < N; i++){
        int c = mp[S[i]];
        if(!has[c]){
            has[c] = true;
            cnt++;
            if(cnt == 4){
                fill(has, has+4, false);
                segs.push_back(i);
                cnt = 0;
            }
        }
    }

    for(int i : segs)
        printf("%c", S[i]);
    for(int i = 0; i < 4; i++){
        if(!has[i]){
            printf("%c\n", ch[i]);
            return 0;
        }
    }
}
```

Swap Game

```
#include
```

```

using namespace std;

const int N = 9;
const int x[] = {0, 1, 2, 3, 4, 5, 1, 2, 4, 5, 7, 8};
const int y[] = {3, 4, 5, 6, 7, 8, 0, 1, 3, 4, 6, 7};

unordered_map dist;
queue Q;
string perm;

void init(){
    perm = "123456789";

    dist[perm] = 1;
    Q.push(perm);
    while(!Q.empty()){
        perm = Q.front(); Q.pop();
        int d = dist[perm];
        for(int i = 0; i < 12; i++){
            swap(perm[x[i]], perm[y[i]]);
            if(!dist[perm]){
                dist[perm] = d+1;
                Q.push(perm);
            }
            swap(perm[x[i]], perm[y[i]]);
        }
    }
}

int main(){
    init();
    for(int i = 0; i < N; i++)
        scanf("%c", &perm[i]);
    printf("%d\n", dist[perm]-1);
}

```

Multiplication Table

```

#include

using namespace std;
typedef long long ll;

ll N;

ll f(ll x){
    ll cnt = 0;
    for(ll i = 1; i <= N; i++)
        cnt += min(N, x/i);
    return cnt;
}

int main(){
    scanf("%lld", &N);
    ll lo = 0, hi = N*N;
    while(hi-lo > 1){
        ll mid = lo + (hi-lo)/2;
        if(f(mid) < (N*N+1)/2) lo = mid;
        else hi = mid;
    }
}

```

```
printf("%lld\n", lo+1);  
}
```

Bubble Sort Rounds I

```
#include  
  
using namespace std;  
typedef pair pii;  
const int maxN = 2e5+1;  
  
int N, x[maxN];  
vector sorted;  
map inv;  
  
int main(){  
    scanf("%d", &N);  
    for(int i = 0; i < N; i++){  
        scanf("%d", &x[i]);  
        sorted.push_back({x[i], i});  
    }  
    sort(sorted.begin(), sorted.end(), greater());  
  
    int worst_dist = 0;  
    for(int i = 0; i < N; i++){  
        int targ = N-i-1;  
        int start = sorted[i].second;  
        worst_dist = max(worst_dist, start - targ);  
    }  
    printf("%d\n", worst_dist);  
}
```

Advertisement

```
#include  
  
using namespace std;  
typedef long long ll;  
const int maxN = 2e5+5;  
  
int N, l[maxN], r[maxN];  
ll best, a[maxN];  
  
int main(){  
    scanf("%d", &N);  
    for(int i = 1; i <= N; i++){  
        scanf("%lld", &a[i]);  
    }  
  
    for(int i = 1; i <= N; i++){  
        l[i] = i-1;  
        while(a[l[i]] >= a[i])  
            l[i] = l[l[i]];  
    }  
}
```

```

    }
    for(int i = N; i >= 1; i--){
        r[i] = i+1;
        while(a[r[i]] >= a[i])
            r[i] = r[r[i]];
    }

    for(int i = 1; i <= N; i++)
        best = max(best, (r[i]-l[i]-1)*a[i]);
    printf("%lld\n", best);
}

```

Special Substrings

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, M;
char S[maxN];
bool exists[26];
map ch;
map, ll> dp;

bool containsEach(vector V){
    for(int i = 0; i < (int) V.size(); i++){
        if(V[i] <= 0)
            return false;
    }
    return true;
}

int main(){
    scanf("%s", S);
    N = (int) strlen(S);
    for(int i = 0; i < N; i++){
        int c = (int) (S[i] - 'a');
        if(!exists[c]){
            ch[S[i]] = M++;
            exists[c] = true;
        }
    }

    ll ans = 0;
    vector freq(M);
    dp[freq]++;
    for(int i = 0; i < N; i++){
        int c = ch[S[i]];
        freq[c]++;
        if(containsEach(freq))
            for(int j = 0; j < M; j++)
                freq[j]--;
        ans += dp[freq];
        dp[freq]++;
    }
    printf("%lld\n", ans);
}

```

Counting LCM Arrays

```
#include

using namespace std;
typedef long long ll;
typedef pair pii;
const ll MOD = 1e9+7;

struct Matrix2x2 {
    ll a, b, c, d;
    Matrix2x2(ll a, ll b, ll c, ll d) : a(a), b(b), c(c), d(d) {}

    Matrix2x2 operator*(const Matrix2x2& other) const {
        return Matrix2x2(
            (a * other.a + b * other.c) % MOD,
            (a * other.b + b * other.d) % MOD,
            (c * other.a + d * other.c) % MOD,
            (c * other.b + d * other.d) % MOD
        );
    }
};

Matrix2x2 fast_pow(Matrix2x2 base, ll exp){
    Matrix2x2 result(1, 0, 0, 1);
    while(exp > 0){
        if(exp % 2 == 1)
            result = result * base;
        base = base * base;
        exp /= 2;
    }
    return result;
}

vector prime_factor(int k){
    vector factors;
    for(int i = 2; i * i <= k; i++){
        int count = 0;
        while(k % i == 0){
            k /= i;
            count++;
        }
        if(count > 0)
            factors.push_back({i, count});
    }
    if(k > 1)
        factors.push_back({k, 1});
    return factors;
}

ll solve(int n, int k){
    vector factors = prime_factor(k);

    ll ans = 1;
    for(const auto& [p, e] : factors){
        Matrix2x2 base(1, 1, e, 0);
        Matrix2x2 result = fast_pow(base, n+1);
        ans = (ans * result.a) % MOD;
    }
    return ans;
}

int main(){
    int T;
```

```
scanf("%d", &T);
for(int t = 0, n, k; t < T; t++){
    scanf("%d %d", &n, &k);
    printf("%lld\n", solve(n, k));
}
}
```

Square Subsets

```
#include

using namespace std;
const int maxX = 5005;
const int SIZE = 700;
const int MOD = 1e9+7;

int N, ans;
bool p[maxX];
vector primes;
bitset basis[700];

void generate_primes(){
    for(int i = 2; i < maxX; i++) p[i] = true;
    for(int i = 2; i < maxX; i++){
        if(p[i]){
            primes.push_back(i);
            for(int j = 2*i; j < maxX; j += i)
                p[j] = false;
        }
    }
}

bitset prime_factor(int x){
    bitset factors;
    for(int i = 0; i < (int) primes.size() && primes[i] <= x; i++){
        int power = 0;
        while(x % primes[i] == 0){
            x /= primes[i];
            power++;
        }
        factors[i] = (power % 2 == 1);
    }
    return factors;
}

bool insert_vector(bitset mask){
    for(int i = SIZE-1; i >= 0; i--){
        if(mask[i] == 0) continue;
        if(!basis[i].any()){
            basis[i] = mask;
            return true;
        }
        mask ^= basis[i];
    }
    return false;
}

int main(){
    scanf("%d", &N);
    generate_primes();
```

```

int ans = 1;
for(int i = 0, x; i < N; i++){
    scanf("%d", &x);
    bitset factors = prime_factor(x);
    if(!insert_vector(factors))
        ans = (ans * 2) % MOD;
}
printf("%d\n", ans);
}

```

Stack Weights

```

#include

using namespace std;
const int maxN = 2e5+1, SIZE = 4*maxN;
const int INF = 0x3f3f3f3f;

int N, lo[SIZE], hi[SIZE], delta[SIZE], mx[SIZE], mn[SIZE];

void push(int i){
    if(delta[i] != 0){
        delta[2*i] += delta[i];
        delta[2*i+1] += delta[i];
        delta[i] = 0;
    }
}

void pull(int i){
    mn[i] = min(mn[2*i]+delta[2*i], mn[2*i+1]+delta[2*i+1]);
    mx[i] = max(mx[2*i]+delta[2*i], mx[2*i+1]+delta[2*i+1]);
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r) return;
    int m = l+(r-l)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void increment(int i, int l, int r, int v){
    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        delta[i] += v; return;
    }

    push(i);
    increment(2*i, l, r, v);
    increment(2*i+1, l, r, v);
    pull(i);
}

int minimum(int i, int l, int r){
    if(l > hi[i] || r < lo[i]) return INF;
    if(l <= lo[i] && hi[i] <= r) return mn[i]+delta[i];

    push(i);
    int lmin = minimum(2*i, l, r);
    int rmin = minimum(2*i+1, l, r);
}

```

```

    pull(i);

    return min(lmin, rmin);
}

int maximum(int i, int l, int r){
    if(l > hi[i] || r < lo[i])    return -INF;
    if(l <= lo[i] && hi[i] <= r)    return mx[i]+delta[i];

    push(i);
    int lmax = maximum(2*i, l, r);
    int rmax = maximum(2*i+1, l, r);
    pull(i);

    return max(lmax, rmax);
}

void place_coin(int c, int s){
    int v = (s == 1 ? -1 : 1);
    increment(1, 1, c, v);
}

char query_stacks(){
    int totmin = minimum(1, 1, N);
    int totmax = maximum(1, 1, N);
    if(totmin >= 0 && totmax <= 0)    return '?';
    else if(totmin >= 0)                return '<';
    else if(totmax <= 0)                return '>';
    else                                return '?';
}

int main(){
    scanf("%d", &N);
    init(1, 1, N);
    for(int i = 0, c, s; i < N; i++){
        scanf("%d %d", &c, &s);
        place_coin(c, s);
        printf("%c\n", query_stacks());
    }
}

```

Pyramid Array

```

#include

using namespace std;
typedef pair pii;
const int maxN = 2e5+1;

int N, ds[maxN];
pii X[maxN];

void update(int idx){
    for(int i = idx; i <= N; i += -i&i) ds[i]++;
}

int query(int idx){
    int cnt = 0;
    for(int i = idx; i; i -= -i&i)
        cnt += ds[i];
    return cnt;
}

```



```

}

int main(){
    scanf("%d", &N);
    for(int i = 0, x; i < N; i++){
        scanf("%d", &x);
        X[i] = {x, i+1};
    }
    sort(X, X+N, [](pii a, pii b){
        return a.first > b.first;
    });

    long long ans = 0;
    for(int k = 0; k < N; k++){
        int idx = X[k].second;
        int l = query(idx);
        ans += min(l, k-l);
        update(idx);
    }
    printf("%lld\n", ans);
}

```

Bit Inversions

```

#include

using namespace std;
const int maxN = 2e5+5;
const int SIZE = 4*maxN;

int N, M, mp[maxN], lo[SIZE], hi[SIZE], dp[3][SIZE];
char S[maxN];

int len(int i){
    return hi[i]-lo[i]+1;
}

int combine(int a, int b){
    return ((a<0)^(b<0)) ? a : a+b;
}

bool allsame(int i){
    return abs(dp[2][i]) == len(i);
}

void pull(int i){
    dp[0][i] = (allsame(2*i) ? combine(dp[0][2*i], dp[0][2*i+1]) : dp[0][2*i]);
    dp[1][i] = (allsame(2*i+1) ? combine(dp[1][2*i+1], dp[1][2*i]) : dp[1][2*i+1]);
    dp[2][i] = max(abs(combine(dp[1][2*i], dp[0][2*i+1])), max(dp[2][2*i], dp[2][2*i+1]));
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        mp[l] = i;
        dp[0][i] = dp[1][i] = (S[l-1] == '0' ? -1 : 1);
        dp[2][i] = abs(dp[0][i]);
        return;
    }
    int m = (l+r)/2;
    init(2*i, l, m);

```

```

    init(2*i+1, m+1, r);
    pull(i);
}

void update(int idx){
    int i = mp[idx];
    dp[0][i] *= -1;
    dp[1][i] = dp[0][i];

    i >>= 1;
    while(i){
        pull(i);
        i >>= 1;
    }
}

int query(){
    return dp[2][1];
}

int main(){
    scanf("%s %d", S, &M);
    N = (int) strlen(S);
    init(1, 1, N);
    for(int i = 0, x; i < M; i++){
        scanf("%d", &x);
        update(x);
        printf("%d%c", query(), (" \n")[i==M-1]);
    }
}

```

Writing Numbers

```

#include

using namespace std;
typedef long long ll;

ll p[19];

void init(){
    p[0] = 1;
    for(int i = 1; i <= 18; i++)
        p[i] = p[i-1] * 10;
}

ll check(ll N){
    ll cnt = 0, l = N, r = 0;
    for(int i = 0; l; i++){
        int d = l % 10;
        l /= 10;
        cnt += (d == 1 ? l*p[i]+r+1 : (l+(d!=0))*p[i]);
        r += p[i] * d;
    }
    return cnt;
}

int main(){
    ll K;
    scanf("%lld", &K);
    ll lo = 1, hi = 1e18;

```

```

init();
while(hi-lo > 1){
    ll mid = lo+(hi-lo)/2;
    if(check(mid) <= K) lo = mid;
    else hi = mid;
}
printf("%lld\n", lo);
}

```

Maximum Building I

```

#include

using namespace std;
const int maxN = 1002;

int N, M, best, h[maxN], l[maxN], r[maxN];
char S[maxN][maxN];

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < N; i++)
        scanf(" %s", S[i]);

    h[0] = h[M+1] = -1;
    for(int i = 1; i <= N; i++){
        for(int j = 1; j <= M; j++){
            h[j] = (S[i-1][j-1] == '*' ? 0 : h[j]+1);

            int smaller;
            for(int j = 1; j <= M; j++){
                smaller = j-1;
                while(h[smaller] >= h[j])
                    smaller = l[smaller];
                l[j] = smaller;
            }
            for(int j = M; j >= 1; j--){
                smaller = j+1;
                while(h[smaller] >= h[j])
                    smaller = r[smaller];
                r[j] = smaller;
            }

            for(int j = 1; j <= M; j++){
                int base = r[j]-l[j]-1;
                best = max(best, base * h[j]);
            }
        }

        printf("%d\n", best);
    }
}

```

Sorting Methods

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+5;

int N, x[maxN], ds[maxN];
ll bit[maxN], ans[4];
set S;
set::iterator it;

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

void update(int idx, int val){
    for(int i = idx; i < maxN; i += -i&i)
        bit[i] += val;
}

ll query(int idx){
    int sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum += bit[i];
    return sum;
}

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf("%d", &x[i]);

        int K = N;
        ans[3] = N;
        fill(ds+1, ds+N+1, -1);
        for(int i = 1; i <= N; i++){
            ans[0] += (i-query(x[i])-1);
            update(x[i], 1);

            if(!merge(i, x[i]))
                ans[1] += (-ds[find(i)]-1);

            S.insert(x[i]);
            it = S.lower_bound(x[i]);
            if(++it != S.end())
                S.erase(it);

            if(x[N-i+1] == K){
                K--;
                ans[3]--;
            }
        }
        ans[2] = N - (int) S.size();

        for(int i = 0; i < 4; i++)
            printf("%lld%c", ans[i], (" \n")[i==3]);
    }
}

```

Cyclic Array

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, last[maxN];
ll K, x[2*maxN];

int main(){
    scanf("%d %lld", &N, &K);
    for(int i = 0; i < N; i++){
        scanf("%lld", &x[i]);
        x[i+N] = x[i];
    }

    int r = 0;
    ll sum = 0;
    for(int l = 0; l < N; l++){
        while(r < l+N && sum + x[r] <= K){
            sum += x[r];
            r++;
        }
        last[l] = r % N;
        sum -= x[l];
    }

    int opt = 0;
    for(int i = 0; i < N; i++)
        opt = last[opt];

    int cnt = 1;
    sum = x[opt];
    for(int i = opt+1; i < N+opt; i++){
        if(sum + x[i] <= K) sum += x[i];
        else {
            sum = x[i];
            cnt++;
        }
    }

    printf("%d\n", cnt);
}
```

List of Sums

```
#include

using namespace std;
typedef long long ll;
const int maxN = 101, maxM = 4951;

int N, M;
```

```
multiset Bvals;
ll A[maxN], B[maxM];

bool check(ll a0){
    A[0] = a0;
    Bvals.clear();
    for(int i = 0; i < M; i++){
        Bvals.insert(B[i]);

        for(int i = 1; i < N; i++){
            for(int j = 0; j < i-1; j++){
                if(Bvals.find(A[j]+A[i-1]) == Bvals.end())
                    return false;
                Bvals.erase(Bvals.lower_bound(A[j]+A[i-1]));
            }
            A[i] = *Bvals.begin() - A[0];
        }

        // Confirm that the last element, A[N-1], works. We desire
        // the sum with the other A[i] to produce the remaining Bvals
        for(int j = 0; j < N-1; j++){
            if(Bvals.find(A[j]+A[N-1]) == Bvals.end())
                return false;
            Bvals.erase(Bvals.lower_bound(A[j]+A[N-1]));
        }

        return true;
    }
}

int main(){
    scanf("%d", &N);
    M = N*(N-1)/2;
    for(int i = 0; i < M; i++){
        scanf("%lld", &B[i]);
    }
    sort(B, B+M);

    for(int i = 1; i < M; i++){
        ll a0 = (B[0] + B[1] - B[i])/2;
        if(1 <= a0 && a0 <= B[0]/2 && check(a0)){
            for(int j = 0; j < N; j++){
                printf("%lld%c", A[j], (" \n")[j==N-1]);
            }
            return 0;
        }
    }

    printf("IMPOSSIBLE\n");
}
```